

1 - Bookmark Job Listings

Now we're going to work on the bookmarking functionality. We have a button on the job details page that allows users to add the job listing to their bookmarks or saved jobs. This is so they can easily access it later on. There will be a link in the navigation to see all the user's bookmarks. We also want to have a remove bookmark button on the details page if the job is already bookmarked.

We're going to accomplish this by creating a new migration to create what we call a pivot table. Its purpose is to relate one resource to another. In this case users to job listings. The table will be called `user_job_bookmarks`. We'll also create a seeder to quickly add bookmarks to the test user.

2 - Job User Bookmarks Migration

Now that we have total CRUD functionality for jobs and authentication with profiles, we are going to move on to bookmarking functionality where users can save or bookmark job listings that they are interested in.

This will require a new migration and table in the database, and a new controller, views. We also need to add new relationships. we will also add some seed data.

The table that we are creating with the migration is called a `pivot table` because it is a table that is used to join two other tables together. It is a table that is used to store the relationship between two other tables.

Earlier we created a `save` method in the `JobController`. You can delete that and delete the route that goes with it in the `routes/web.php` file. I mean you could use the Job controller for this stuff but I think it is more organized to have a separate controller for this.

Add Relationships

We talked about relationships a while ago. We added a one to many relationship between jobs and users. Now we need a many to many relationship between jobs and users when it comes to bookmarks. A user can bookmark many jobs, and a job can be bookmarked by many users.

Open the `/app/Models/Job.php` file and add the following import:

```
use Illuminate\Database\Eloquent\Relations\BelongsToMany;
```

Add the following method to the `Job` model:

```
public function bookmarkedByUsers(): BelongsToMany
{
    return $this->belongsToMany(User::class, 'job_user_bookmarks')-
>withTimestamps();
}
```

This will allow us to do things like `$job->bookmarkedByUsers` to get all the users that have bookmarked a job.

Open the `/app/Models/User.php` file and add the following import:

```
use Illuminate\Database\Eloquent\Relations\BelongsToMany;
```

Add the following method to the `User` model:

```
public function bookmarkedJobs(): BelongsToMany
{
    return $this->belongsToMany(Job::class, 'job_user_bookmarks')-
    >withTimestamps();
}
```

This will allow us to do things like `$user->bookmarkedJobs` to get all the jobs that a user has bookmarked.

We have the relationships set up, but we need to add the pivot table to the database.

Create a New Migration

Let's start with the database migration. Run the following command to create a new migration:

```
php artisan make:migration create_job_user_bookmarks_table
```

Open the `/database/migrations/DATE_create_job_user_bookmarks_table.php` file and add the following code:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    public function up(): void
    {
        Schema::create('job_user_bookmarks', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->onDelete('cascade');
            $table->foreignId('job_id')->constrained('job_listings')-
            >onDelete('cascade');
            $table->timestamps();
        });
    }
}
```

```
public function down(): void
{
    Schema::dropIfExists('job_user_bookmarks');
}
};
```

We are creating a new table called `job_user_bookmarks` with two foreign keys, `user_id` and `job_id`. The `user_id` foreign key references the `id` column in the `users` table, and the `job_id` foreign key references the `id` column in the `jobs` table. We are also adding timestamps to the table.

Run the migration:

```
php artisan migrate
```

Check the database by using PG Admin, the command line or another tool and you should see the new table.

3 - Seeding Bookmarks

Before we move on, let's make it so that when we run our seeder, it adds random job listings as bookmarks for our test user.

Return The Test user

First, we need to return the user from the `/database/seeder/TestUserSeeder.php` file:

```
public function run(): any
{
    $user = User::create([
        'name' => 'Test User',
        'email' => 'test@test.com',
        'email_verified_at' => Carbon::now(),
        'password' => Hash::make('12345678'),
    ]);

    return $user;
}
```

Bookmark Seeder

Let's now create a bookmark seeder. Run the following command to generate a new seeder:

```
php artisan make:seeder BookmarkSeeder
```

Open the `database/seeder/BookmarkSeeder.php` file and add the following imports:

```
use App\Models\User;
use App\Models\Job;
```

Now add the following `run` method:

```
public function run(): void
{
    // Get the test user
    $testUser = User::where('email', 'test@test.com')->firstOrFail();

    // Get all job IDs
    $jobIds = Job::pluck('id')->toArray();

    // Randomly select job IDs to bookmark
    $randomJobIds = array_rand($jobIds, 3); // Change 3 to however many you want
    to bookmark

    // Attach the selected jobs as bookmarks for the test user
    foreach ($randomJobIds as $jobId) {
        $testUser->bookmarkedJobs()->attach($jobIds[$jobId]);
    }
}
```

Update `DatabaseSeeder.php`

Open the `database/seeder/DatabaseSeeder.php` file and truncate the new `job_user_bookmarks` table and run the `BookmarkSeeder` seeder:

```
public function run(): void
{
    // Truncate tables
    DB::table('job_listings')->truncate();
    DB::table('users')->truncate();
    DB::table('job_user_bookmarks')->truncate();

    $this->call(TestUserSeeder::class);
    $this->call(RandomUserSeeder::class);
    $this->call(JobSeeder::class);
    $this->call(BookmarkSeeder::class);
}
```

Now run the following command to seed the database:

```
php artisan db:seed
```

You should now see the test user has 3 random job listings bookmarked.

4 - Get & Show Bookmarks

We have our table in the database, now let's add the controller and routes to get the bookmarks and the view to show them.

Bookmark Route

Open the `routes/web.php` file and add the following route grouped with the auth middleware:

```
Route::middleware('auth')->group(function () {
    Route::get('/bookmarks', [BookmarkController::class, 'index'])->name('bookmarks.index');
});
```

be sure to import the `BookmarkController` class:

```
use App\Http\Controllers\BookmarkController;
```

Bookmark Controller

Open a terminal and type the following command to generate a new controller:

```
php artisan make:controller BookmarkController
```

Now open the `app/Http/Controllers/BookmarkController.php` file and add the following imports:

```
use App\Models\Job;
use Illuminate\Http\RedirectResponse;
use Illuminate\View\View;
use Illuminate\Support\Facades\Auth;
```

Now add the following `index` method:

```
public function index(): View
{
    $user = Auth::user();
```

```
$bookmarks = $user->bookmarkedbookmarks()->paginate(10);  
dd($jobs);
```

The `bookmarkedJobs` pertains to the method we created in the `User` model. The `paginate(10)` method will paginate the results.

Let's test this by going to the `bookmarks` route in the browser. You should see a dump of the object with the collection of bookmarks.

Let's return a view instead:

```
// @desc Show all bookmarks for user  
// @route GET bookmarks  
public function index(): View  
{  
    $user = Auth::user();  
  
    $bookmarks = $user->bookmarkedJobs()->paginate(10);  
  
    return view('jobs.bookmarked')->with('bookmarks', $bookmarks);  
}
```

Show Bookmarks in View

Create a file at `/resources/views/jobs/bookmarked.blade.php` and add the following:

```
<x-layout>  
    <h2 class="text-center text-3xl mb-4 font-bold border border-gray-300 p-3">  
        Bookmarked Jobs  
    </h2>  
    <div class="grid grid-cols-1 md:grid-cols-3 gap-4 mb-6">  
        @forelse ($bookmarks as $bookmark)  
            <x-job-card :job="$bookmark" />  
        @empty  
            <p class="text-center text-gray-500">No bookmarks available</p>  
        @endforelse  
    </div>  
  
    {{ $bookmarks->links() }}  
</x-layout>
```

You should now see the cards for the jobs that you bookmarked.

6 - Bookmarking Jobs

We can now see the jobs that we have created. Now we need to be able to bookmark them.

Add Bookmark Route

Open the routes file and add the post route in the group:

```
Route::middleware('auth')->group(function () {
    Route::get('/bookmarks', [BookmarkController::class, 'index'])-
>name('bookmarks.index');
    Route::post('/bookmarks/{job}', [BookmarkController::class, 'store'])-
>name('bookmarks.store');
});
```

store Method

Let's add the `store` method to our `Job` controller. This method will allow us to bookmark a job.

```
// @desc    Store a bookmark
// @route   POST /bookmarks/{job}
public function store(Job $job): RedirectResponse
{
    $user = Auth::user();

    // Check if the job is already bookmarked
    if ($user->bookmarkedJobs()->where('job_id', $job->id)->exists()) {
        return back()->with('status', 'Job is already bookmarked.');
```

```
    }

    // Create a new bookmark
    $user->bookmarkedJobs()->attach($job->id);

    return back()->with('status', 'Job bookmarked successfully.');
```

We are first checking if the job is already bookmarked. If it is, we are returning a message saying so. If it is not, we are creating a new bookmark.

Bookmark Button

Let's open the `resources/views/jobs/show.blade.php` file. Right now there is just a link for the add bookmark button. We need to change it to a form. Replace it with the following:

```
<!-- Bookmark Button -->
@guest
<p
    class="mt-10 bg-gray-200 text-gray-700 font-bold w-full py-2 px-4 rounded-full
text-center"
>
    <i class="fas fa-info-circle mr-3"></i> You must be logged in to bookmark this
```

```

    job.
</p>
@else
<form
    action="{ route('bookmarks.store', $job->id) }"method="POST" class="mt-10">
    @csrf
    <button
        type="submit"
        class="bg-blue-500 hover:bg-blue-600 text-white font-bold w-full py-2 px-4
rounded-full flex items-center justify-center"
    >
        <i class="fas fa-bookmark mr-3"></i> Bookmark Listing
    </button>
</form>
@endguest

```

If the user is not logged in, we are showing a message saying so. If the user is logged in, we are showing a form/button.

We should now be able to bookmark jobs and see them on the saved page. In the next lesson we will add the functionality to remove bookmarks.

7 - Removing bookmarks

Now, we want to be able to remove the bookmarks. The heavy lifting is done. We already have the button submitting to the `destroys` route. We just need to add the functionality to remove the bookmark.

Delete Bookmark Route

Open the routes file and add the delete route in the group:

```

Route::middleware('auth')->group(function () {
    Route::get('/bookmarks', [BookmarkController::class, 'index'])-
>name('bookmarks.index');
    Route::post('/bookmarks/{job}', [BookmarkController::class, 'store'])-
>name('bookmarks.store');
    Route::delete('/bookmarks/{job}', [BookmarkController::class, 'destroy'])-
>name('bookmarks.destroy');
});

```

Open the `app/Http/Controllers/BookmarkController.php` file. Add the following code for the `destroy` method:

```

// @desc    Remove a bookmark
// @route    DELETE /bookmarks/{job}
public function destroy(Job $job): RedirectResponse
{

```



```

$user = Auth::user();

// Check if the job is bookmarked before trying to remove it
if (!$user->bookmarkedJobs()->where('job_id', $job->id)->exists()) {
    return back()->with('error', 'Job is not bookmarked.');
```

```

}

// Remove the bookmark
$user->bookmarkedJobs()->detach($job->id);
```

```

return back()->with('status', 'Job removed from bookmarks successfully.');
```

We check if the user is logged in. If they are not, we redirect them back to the previous page with a message. If they are logged in, we detach the bookmark from the user and redirect them back to the previous page with a message.

Edit Button

We want to show a remove link/form if the user already has the job bookmarked. Add the following to the `resources/views/jobs.show.blade.php` file:

```

<!-- Bookmark Button -->
@guest
<p
    class="mt-10 bg-gray-200 text-gray-700 font-bold w-full py-2 px-4 rounded-full
    text-center"
>
    <i class="fas fa-info-circle mr-3"></i> You must be logged in to bookmark this
    job.
</p>
@else
<form
    action="{ { auth()->user()->bookmarkedJobs()->where('job_id', $job->id)->exists()
? route('bookmarks.destroy', $job->id) : route('bookmarks.store', $job->id) }}"
    method="POST"
    class="mt-10"
>
    @csrf
    @if (auth()->user()->bookmarkedJobs()->where('job_id', $job->id)->exists())
        @method('DELETE')
        <button
            type="submit"
            class="bg-red-500 hover:bg-red-600 text-white font-bold w-full py-2 px-4
            rounded-full flex items-center justify-center"
        >
            <i class="fas fa-bookmark mr-3"></i> Remove Bookmark
        </button>
    @else
        <button
```

```
      type="submit"
      class="bg-blue-500 hover:bg-blue-600 text-white font-bold w-full py-2 px-4
rounded-full flex items-center justify-center"
    >
      <i class="fas fa-bookmark mr-3"></i> Bookmark Listing
    </button>
  @endif
</form>
@endguest
```

We are checking if the bookmark exists and if it does, use the destroy route and change the text and styles.